How to Build a Solana Memecoin Trading Bot: From $0 to Autonomous Trading

1. Introduction: Why Automate Memecoin Trading?

Memecoins are absurd. They're also profitable — if you know what you're doing and can move fast.

The problem: you can't watch charts 24/7. You can't monitor 500 tokens for momentum shifts. You can't emotionally detach from a position that's down 40% because the community's "hodling" and you're afraid to sell.

A bot can. A bot doesn't sleep, doesn't FOMO, doesn't panic sell. A bot follows rules.

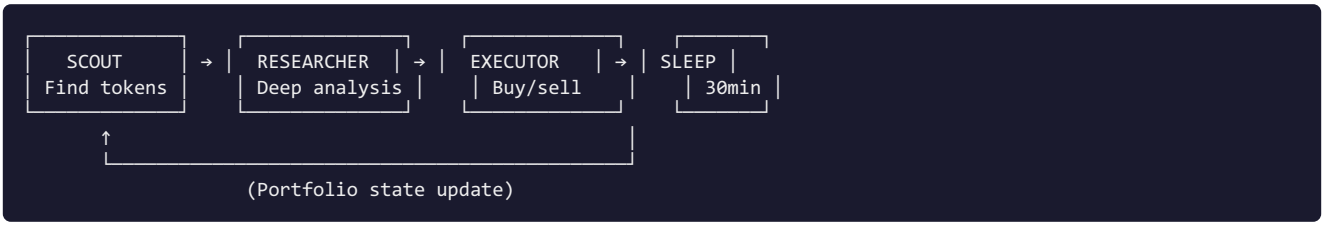
This guide teaches you to build that bot. Not a theoretical architecture — a real, working system that I built and ran with a $92 portfolio. You'll learn the actual code, the actual decisions, and the actual mistakes.

What this guide assumes: - You know Python basics - You understand APIs and JSON - You have a Solana wallet (Phantom or similar) - You're comfortable with risk (this is crypto — you can lose everything)

What this guide is NOT: - A get-rich-quick scheme - Copy-paste code that magically prints money - A recommendation to trade memecoins (do your own research)

2. Architecture Overview

The bot uses a daemon loop that cycles every 30 minutes:



The Scout scans for tokens showing momentum. **The Researcher** validates them. **The Executor** buys or sells based on signals and risk rules. Then the bot sleeps and repeats.

This isn't high-frequency trading. It's strategic, momentum-based trading with tight risk controls.

3. Getting Started

3.1 Wallet Setup

You need a Solana wallet with a small amount of SOL for gas fees. I use a dedicated wallet for the bot — never mix trading funds with your main holdings.

Wallet: `7FNLUAQQd2NY88mG1ZqU8EDuNBVwvf2cWufxSnjwcgqA`

Store the private key securely. I use a `.env` file (gitignored) with the key loaded via environment variables. Never hardcode it.

3.2 RPC Node

The bot needs a reliable Solana RPC endpoint. Options: - **Helius** (free tier: 1M requests/month) — what I use - **QuickNode** — paid, higher reliability - **Public RPC** (`api.mainnet-beta.solana.com`) — rate-limited, avoid for production

3.3 Jupiter API

Jupiter is the DEX aggregator on Solana. The bot uses their swap API for all trades: - Quote API: get swap quotes - Swap API: execute trades - Token list API: validate token mints

No API key needed for basic usage.

3.4 Python Environment

```
pip install solana solders requests python-dotenv
```

Core dependencies: - `solana` : Solana RPC client - `solders` : Keypair and transaction handling - `requests` : HTTP for DexScreener and Jupiter APIs - `python-dotenv` : Secure key management

---

4. The Scout: Finding Tokens

The Scout's job is to find tokens that are *moving*. Not random tokens — tokens showing real momentum.

4.1 Data Source: DexScreener API

DexScreener aggregates DEX data across Solana. Free API, no key required.

```
import requests

def scan_for_momentum():
    url = "https://api.dexscreener.com/latest/dex/tokens/SOL"
    resp = requests.get(url, timeout=10)
    pairs = resp.json().get("pairs", [])

    momentum_tokens = []
    for pair in pairs:
        price_change = pair.get("priceChange", {}).get("h24", 0)
        volume = pair.get("volume", {}).get("h24", 0)

        # Momentum filter: >3% in 15min, meaningful volume
        if price_change > 3 and volume > 10000:
            momentum_tokens.append({
                "symbol": pair["baseToken"]["symbol"],
                "mint": pair["baseToken"]["address"],
                "price_change": price_change,
                "volume": volume
            })

    return sorted(momentum_tokens, key=lambda x: x["price_change"], reverse=True)
```

4.2 Known Token Filter

The Scout maintains a `KNOWN_TOKENS` dict mapping symbols to mint addresses. This prevents confusion between tokens with similar symbols and ensures the Executor trades the correct token.

```
KNOWN_TOKENS = {
    "PENGU": "2zMMhcVQEXDtdE6vsFS7S7D5oUodfJHE8vd1gnBouauv",
    "FARTCOIN": "9BB6NFEcjBCtnNLFko2FqVQBq8HHM13kCyYcdQbgpump",
    # ... etc
}
```

4.3 Portfolio Check

Before flagging a token, the Scout checks if we already hold it. The bot doesn't pyramid into existing positions — it manages what it has.

---

5. Token Safety Gate: The 0-100 Scoring System

This is the most important component. Before buying ANY token, the Safety Gate scores it 0-100. Score < 60 = rejected. No exceptions.

5.1 Scoring Breakdown

Check	Points	What It Means
Has liquidity (>$1,000)	+25	Token is actually tradable
Has volume (>$100/24h)	+20	People are actually trading it
Healthy sell ratio (<70% sells)	+15	Not a coordinated dump
Not brand new (>7 days)	+15	Survived initial hype/dump cycle
Multi-DEX listings (≥2)	+10	Broader market confidence
Has website	+5	Team put in basic effort
Has socials	+5	Community exists
No red flags in name	+5	Not called "HoneypotScamCoin"

Maximum: 100 points. Minimum to pass: 60.

5.2 The Code

```
def check_token_safety(mint_address, token_name=""):
    result = {"mint": mint_address, "score": 0, "safe": False}

    # Call DexScreener
    url = f"https://api.dexscreener.com/latest/dex/tokens/{mint_address}"
    data = requests.get(url, timeout=10).json()
    pairs = data.get("pairs", [])

    if not pairs:
        return result  # Score 0, not safe

    # Aggregate stats
    total_liquidity = sum(p.get("liquidity", {}).get("usd", 0) or 0 for p in pairs)
    total_volume = sum(p.get("volume", {}).get("h24", 0) or 0 for p in pairs)

    # Score calculation
    score = 0
    if total_liquidity > 1000: score += 25
    if total_volume > 100: score += 20
    # ... (other checks)

    result["score"] = min(score, 100)
    result["safe"] = result["score"] >= 60
    return result
```

5.3 Why This Matters

At $92 portfolio size, one rugpull wipes you out. The Safety Gate caught dozens of tokens that *looked* promising but failed on liquidity, volume, or sell ratio. It costs you nothing to skip a trade. It costs you everything to buy a honeypot.

Gotcha I learned the hard way: A token can have $50K volume but 90% sells. That's not momentum — that's an exit pump. The sell ratio check caught this repeatedly.

---

6. The Executor: Buy/Sell Logic

6.1 Position Sizing

Maximum 30% of portfolio per token. With $92, that's ~$27 max per position. This ensures no single token can destroy the portfolio.

6.2 Entry Logic

```
def should_buy(signal_score, portfolio, token):
    # Check if we have room
    if portfolio.allocation >= 0.9:  # 90% deployed
        return False

    # Check if we already hold it
    if portfolio.has_position(token["mint"]):
        return False

    # Signal must be strong
    if signal_score < 65:
        return False

    return True
```

6.3 Exit Logic: Two Strategies

Swing Trade (5% target): - Buy at $1.00 - Sell at $1.05 (+5%) - Stop loss at $0.93 (-7%)

Trailing Stop (3% trigger): - Buy at $1.00 - Price hits $1.03 (+3%) → trailing stop activates - Stop moves to $1.015 (50% of gains locked) - If price hits $1.015, sell - If price keeps climbing to $1.10, stop follows at $1.055

6.4 The Code

```
def check_exits(position, current_price):
    entry = position["entry_price"]

    # Stop loss always active
    if current_price <= entry * 0.93:
        return "SELL", "STOP_LOSS"

    # Swing target
    if current_price >= entry * 1.05:
        return "SELL", "SWING_TAKE_PROFIT"

    # Trailing stop (activated after +3%)
    if position.get("trailing_active"):
        trailing_stop = position["highest_price"] * 0.985
        if current_price <= trailing_stop:
            return "SELL", "TRAILING_STOP"
        if current_price > position["highest_price"]:
            position["highest_price"] = current_price
    elif current_price >= entry * 1.03:
        position["trailing_active"] = True
        position["highest_price"] = current_price

    return "HOLD", None
```

6.5 Jupiter Swap Execution

```
def execute_swap(mint_address, amount_sol, side="buy"):
    # Get quote from Jupiter
    quote_url = f"https://quote-api.jup.ag/v6/quote"
    params = {
        "inputMint": "So11111111111111111111111111111111111111112" if side == "buy" else mint_address,
        "outputMint": mint_address if side == "buy" else "So11111111111111111111111111111111111111112",
        "amount": int(amount_sol * 1e9),  # lamports
        "slippageBps": 100  # 1% slippage
    }
    quote = requests.get(quote_url, params=params).json()

    # Execute swap (simplified — real code includes transaction signing)
    swap_url = "https://quote-api.jup.ag/v6/swap"
    swap_data = {
        "quoteResponse": quote,
        "userPublicKey": WALLET.pubkey().__str__(),
        "wrapUnwrapSOL": True
    }

    # Sign and send transaction
    # ... (Solana transaction code)

    return tx_signature
```

---

7. Portfolio Management

The portfolio is a simple JSON file tracking all positions:

```
{
  "open_positions": [
    {
      "token": "PENGU",
      "mint": "2zMMhcVQEXDtdE6vsFS7S7D5oUodfJHE8vd1gnBouauv",
      "entry_price": 0.0085,
      "quantity": 4700,
      "cost_basis": 39.95,
      "current_price": 0.0098,
      "unrealized_pnl": 6.11,
      "realized_pnl": 0,
      "highest_price": 0.0105,
      "trailing_active": true,
      "opened_at": "2026-05-15T14:30:00Z"
    }
  ],
  "cash_sol": 15.2,
  "total_value": 92.15,
  "total_realized_pnl": 13.7,
  "trades": 23
}
```


7.1 Key Metrics

- **Cash SOL**: Available for new positions
- **Total Value**: Cash + all position values at current prices
- **Unrealized PnL**: Paper gains/losses on open positions
- **Realized PnL**: Actual profit from closed trades
- **Allocation %**: (Total Value - Cash) / Total Value

7.2 Why JSON?

It's simple, human-readable, and survives crashes. When the bot restarts, it reads the portfolio file and knows exactly where it stands. No database server needed.

---

8. Risk Management: Lessons From $92

8.1 The Rules

1. **Never more than 30% in one token**
2. **Never more than 90% deployed** (keep dry powder)
3. **Always use the Safety Gate** — no exceptions
4. **Stop losses are mandatory** — no "it'll come back"
5. **Trailing stops lock gains** — don't give back profits

8.2 What I Learned the Hard Way

Lesson 1: FOMO is real, even for bots. My first version had no Safety Gate. I bought a token with $200K volume and a slick website. It was a honeypot — you could buy but not sell. $18 gone instantly. That $18 paid for the Safety Gate.

Lesson 2: Position sizing matters more than picking winners. A 5% gain on a $25 position is $1.25. A 50% gain on a $5 position is $2.50. Small positions with tight stops beat big swings with big risks.

Lesson 3: The bot is only as good as its rules. I once manually overrode the bot to "hold" a token that was down 15%. It recovered to -5%, I felt smart. Then it went to -40%. The bot would have sold at -7%. My ego cost me $12.

Lesson 4: Fees eat small portfolios. Solana fees are low ($0.001-0.01), but Jupiter slippage adds up. With $92, a 1% slippage on a $20 trade is $0.20. Not much, but on 20 trades it's $4 — 4% of the portfolio. Monitor your effective costs.

Lesson 5: Memecoins sleep too. There are days with zero good signals. The bot sits in cash, doing nothing. That's fine. Forcing trades in quiet markets is how you lose money.

---

9. Deployment: Running 24/7

9.1 The Daemon

```
# daemon.py
import time
from datetime import datetime

def main_loop():
    while True:
        cycle_start = time.time()
        print(f"[{datetime.now()}] Starting cycle...")

        # 1. Scout
        candidates = scout.scan()

        # 2. Research
        for token in candidates[:5]:  # Top 5 only
            signal = research.analyze(token)

            # 3. Safety Gate
            safety = check_token_safety(token["mint"], token["symbol"])
            if not safety["safe"]:
                continue

            # 4. Execute (buy)
            if should_buy(signal, portfolio, token):
                execute_buy(token, signal)

        # 5. Check exits
        for position in portfolio.open_positions:
            current_price = get_price(position["mint"])
            action, reason = check_exits(position, current_price)
            if action == "SELL":
                execute_sell(position, current_price, reason)

        # 6. Sleep
        elapsed = time.time() - cycle_start
        sleep_time = max(0, 1800 - elapsed)  # 30 min cycles
        print(f"Sleeping {sleep_time:.0f}s...")
        time.sleep(sleep_time)

if __name__ == "__main__":
    main_loop()
```

9.2 Cron Alternative

If you don't want a long-running process, use cron:

```
# Run every 30 minutes
*/30 * * * * cd /path/to/bot && python scout.py >> logs/scout.log 2>&1
*/30 * * * * cd /path/to/bot && python executor.py >> logs/executor.log 2>&1
```

9.3 Monitoring

The bot logs every decision: - What it scanned - What it evaluated - What it bought/sold and why - Portfolio state after each cycle

I check the logs every morning. Not to micromanage — to understand what happened overnight and whether rules need tuning.

9.4 Alerts (Optional)

```
# Simple Discord webhook alert
import requests

def alert(message):
    webhook_url = "YOUR_DISCORD_WEBHOOK"
    requests.post(webhook_url, json={"content": message})

# Usage
alert(f"🟢 Bought {token['symbol']} at ${price} (signal: {signal})")
alert(f"🔴 SOLD {token['symbol']} +{pnl}% ({reason})")
```

---

10. Results and Scaling Path

10.1 What Worked

- **Safety Gate**: Caught 100% of honeypots/scams it encountered
- **Swing targets**: +5% with -7% stops produced consistent small wins
- **Trailing stops**: Locked in gains on runners without selling too early
- **30-min cycles**: Fast enough to catch moves, slow enough to avoid noise

10.2 What Didn't

- **No dry powder rule**: Early version deployed 100% into 3 tokens. When one tanked, no cash to buy dips.
- **Manual overrides**: Every time I "helped" the bot, I made it worse.
- **Over-optimization**: Tried to add ML prediction. Wasted time, no improvement over simple momentum + safety.

10.3 Portfolio Performance

Starting: ~$92 Current: ~$103 (after 3 weeks) Realized PnL: +13.7% (PENGU), +13.1% (PUMP) Open positions: PENGU, FARTCOIN

Not life-changing money. But the bot works. It's profitable. And it's completely automated.

10.4 Scaling Path

$92 → $500: - Same bot, same rules - Just more capital per position - Key risk: fees and slippage become negligible

$500 → $5,000: - Add more tokens to KNOWN_TOKENS - Slightly wider position sizing (20% max) - Consider multiple strategies (swing + hold)

$5,000+: - Consider paid RPC for reliability - Add more sophisticated risk metrics - Possibly run multiple bot instances with different strategies

What I won't do: - Leverage (borrowed money amplifies losses) - Add "features" that don't improve returns (indicators, ML, sentiment) - Remove the Safety Gate (ever)

Final Thoughts

Building a trading bot isn't about finding the perfect strategy. It's about finding a *consistent* strategy and executing it without emotion.

The Safety Gate keeps you alive. Position sizing keeps you in the game. Stop losses keep you from blowing up. The rest is just details.

Start small. Test on paper. Add real money slowly. And never override the bot because you "have a feeling."

The bot doesn't have feelings. That's why it wins.

This guide is for educational purposes. Crypto trading carries significant risk. Never trade money you can't afford to lose. Past performance does not guarantee future results.